Formulating Distance Functions via the Kernel Trick

Names removed for double blind review

ABSTRACT

Every information-retrieval task depends on a good distance function for measuring similarity between data instances. The most effective distance function must be formulated in an context-dependent (applicaiton-, data-, and user-dependent) way. In this paper, we propose a novel method, which performs a nonlinear transformation on an existing distance function based on the contextual information provided by the application, data, or user. We show that through a process called the *kernel trick*, this flexible, non-linear transformation can be performed efficiently in a projected space. Theoretically, we demonstrate that our method is optimal. Empirically, using several datasets and applications, we show that our method to be very effective.

1. INTRODUCTION

A critical task of information organization and retrieval is the formulation of application-, data-, user-, and query-dependent distance functions for measuring data similarity. To date, most applications simply employ the *Euclidean distance* for measuring similarity. However, an effective distance function ought to consider the idosyncracy of the application, data, user, and query (hereafter we refer as contextual information) to meaningfully measure similarity. The quality of the distance function significantly affects the success in organizing data or finding meaningful results [2, 3, 10, 12, 14].

How do we consider contextual information in formulating a good distance function? One extension of the popular Euclidean distance (or more generally, the L_p-norm) is to weight the data attributes (features) based on their importance for a target task [12]. For example, for answering a *sunset* image-query, color features should be weighted higher. For answering an *architecture* image-query, shape and texture features may be more important. Several past work in the information retrieval (IR) community and some recent work in the data-mining community [**?**, 3] propose feature-weighting schemes based on contextual information. Weighting the features is equivalent to performing a *linear* transformation in the space formed by the features. Although linear models enjoy the twin advantages of simplicity of description and efficiency of computation, this same simplicity is insufficient to model similarity for many real-world datasets. For example, it has been widely acknowledged in the image/video retrieval domain that a query concept is typically a non-linear combination of perceptual features (color, texture, and shape) [22, 24, **?**]. In this paper we propose performing a *non-linear* transformation on the features to gain greater fleaibility for mapping features to semantics.

We name our method *distance-metric alignment* (DMA). The inputs to DMA are an initial distance metric, and *contextual information*. Contextual information can be conveyed through the form of *training data* or *side information* (discussed in details in Section **??**). For instance, in the biomedical domain, physicians can provide information such as which pairs of proteins may have similar functions. In the information-retrieval domain, Web users can provide relevance feedback to the search engine indicating which returned documents/images are relevant to their queries. Through *side information*, domain knowledge or user preferences are provided for transforming the feature space. The similarity scores of unseen data pairs can then be measured in the transformed space to better reflect the idosyncracy of the application, data, user, and query.

At first it might seem that our non-linear transformations can suffer from high computational complexity. DMA avoids this concern by employing the *kernel trick*[1]. The kernel trick lets us generalize distance-based algorithms to operate in the *projected space* (defined next), usually nonlinearly related to the *input space*. The *input space* (denoted as $\mathcal{I}$) is the original space in which data vectors are located, and the *projected space*[2] (denoted as $\mathcal{P}$) is that space to which the data vectors are projected, linearly or non-linearly. The advantage of using the *kernel trick* is that, instead of explicitly determining the coordinates of the data vectors in the projected space, the distance computation in $\mathcal{P}$ can be efficiently performed in $\mathcal{I}$ through a kernel function. Specifically, given two vectors $\mathbf{x}_i$ and $\mathbf{x}_j$, kernel function $K(\mathbf{x}_i, \mathbf{x}_j)$ is defined as the inner product of $\phi(\mathbf{x}_i)$ and $\phi(\mathbf{x}_j)$, where ϕ is a basis function that maps the vectors $\mathbf{x}_i$ and $\mathbf{x}_j$ from $\mathcal{I}$ to $\mathcal{P}$. The inner product between two vectors can be thought of as a measure of their similarity. Therefore, $K(\mathbf{x}_i, \mathbf{x}_j)$ returns the similarity of $\mathbf{x}_i$ and $\mathbf{x}_j$ in $\mathcal{P}$. The distance between $\mathbf{x}_i$

[1]The *kernel trick* was first published in 1964 in the paper of M. Aizerman, E. Braverman, and L. Rozonoer's [4]. The kernel trick has been applied to several algorithms in statistics, including Support Vector Machines and kernel PCA.

[2]In machine learning community, this projected space is also called feature space [25]

and $\mathbf{x}_j$ in terms of the kernel is defined as

$$d(\mathbf{x}_i, \mathbf{x}_j) = \|\phi(\mathbf{x}_i) - \phi(\mathbf{x}_j)\|_2 = \sqrt{K(\mathbf{x}_i, \mathbf{x}_i) + K(\mathbf{x}_j, \mathbf{x}_j) - 2K(\mathbf{x}_i, \mathbf{x}_j)}. \quad (1)$$

Since a kernel function can be either linear or nonlinear, the traditional feature-weighting approach (e.g., [**?**, 3]) is just a special case of DMA.

1.1 Illustrative Examples

We use three examples to illustrate

We use a simple example to illustrate our method. Figure 1(a) shows a dataset in the input space $\mathcal{I}$ with four different categories, black car (denoted as black solid circle) , green car (denoted as green solid circle), black box (black cross), and green box (green cross). Without any information from the user, by using a given prior distance function, the classification strategy might classify the dataset into the red class and the green class, obtaining a class boundary[3] denoted as the curve in Figure 1(a). However, the user might not intend to classify the red instances out of the green instances. They might be interested in identifying cars out of boxes. Therefore, we have to modify the prior distance function so as to be consistent with the user's query concept. DMA works by first selecting some instances in Figure 1(a) to query the user's subjective concept. As shown in Figure 1(b), the user then provides the feedback as contextual information about whether two instances are similar (denoted as a solid ellipse) or dissimilar (denoted as a semi-solid ellipse). Then, DMA learns a new distance function from contextual information, where the intra-class distance is decreased and the inter-class distance is increased. Finally, DMA applies the learned distance function to the original dataset in $\mathcal{I}$, where the new class boundary is better aligned to the context of the dataset, as shown in Figure 1(c). Notice that different users can label the contextual information differently, and DMA is able to align the boundary in a personalized way.

1.2 Contribution Summary

In this paper, we address a core problem of information management: formulating a context-based distance function to improve the accuracy of similarity measurement, using *distance function alignment* or DMA. In additional to the overall approach, the specific contributions of this work are as follows:

- We propose an efficient method to perform non-linear transformation on a prior distance function for adapting a similarity measure to contextual information (Section 3). We believe that this work is essential for supporting U.S. National Science Foundation's research vision on context-based information retrieval [1].
- We provide the proof of optimality of the DMA algorithm (Section 3.2).
- We empirically demonstrate the effectiveness of DMA on various problems of interest like classification and clustering (Section 4).

[3]Other class boundaries might exit, depending on the prior distance function and the classification strategy applied on the dataset. Here, we suppose that the dataset is classified into the red and the green to illustrate a case where a prior distance function might not result in a boundary at which the user is satisfied.

2. RELATED WORK

Distance function learning approaches can be divided broadly into metric learning and kernel learning approaches. In the rest of this section we discuss representative work of these two approaches.

2.1 Metric Learning

Metric learning attempts to find the optimal linear transformation for the given set of data vectors to better characterize the similarity between vectors. The transformation by itself is linear, but the data vectors may first be *explicitly* mapped to a new set of vectors using a nonlinear function $\phi(\mathbf{x})$. The transformation of the data vectors is equivalent to assigning weights to the features of the vectors; therefore, metric learning is often called *feature weighting*. The metric learning approach is given a set of data vectors $\mathcal{X} = \{\mathbf{x}\}_{i=1}^{m}$ in $\Re^n$ and similarity information in the form of $(\mathbf{x}_i, \mathbf{x}_j) \in S$ (a similar set), if $\mathbf{x}_i$ and $\mathbf{x}_j$ are similar. Metric learning aims to learn a distance metric $d_{\mathbf{M}}(\mathbf{x}_i, \mathbf{x}_j)$ between data vectors $\mathbf{x}_i$ and $\mathbf{x}_j$ that respects the similarity information. Mathematically the distance metric can be represented as

$$d_{\mathbf{M}}(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{(\phi(\mathbf{x}_i) - \phi(\mathbf{x}_j))^T \mathbf{M} (\phi(\mathbf{x}_i) - \phi(\mathbf{x}_j))},$$

where $\mathbf{M}$ needs to be positive (semi-) definite so as to satisfy metric properties—non-negativity and triangle inequality. More generally, $\mathbf{M}$ parameterizes a family of Mahalanobis distances over $\Re^n$. The choice of the basis function ϕ and the scaling matrix $\mathbf{M}$ will differentiate the various metric learning algorithms.

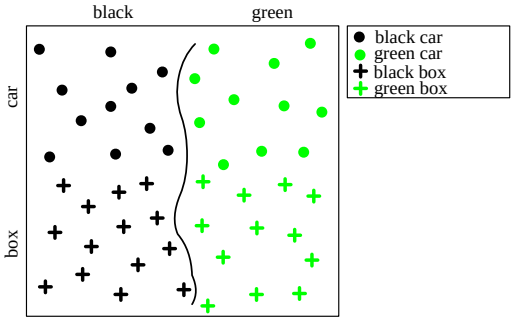
Wettschereck et al. [27] provide a review of the performance of feature-weighting algorithms with emphasis on their performance for the k-nearest neighbor classifier. Here, we discuss only a few representative algorithms. (For the other algorithms, please refer to [27].) A number of papers address the problem of learning distance metrics using contextual information[4] in the form of groups of similar vectors [7, 28]. Contextual information can be user-provided information on the similarity characteristics of a subset of data. Based on this information, the work of [7] uses Relevant Component Analysis to efficiently learn a full rank Mahalanobis metric [21]. The authors use equivalence relations for the contextual information. They compute

$$\hat{C} = \frac{1}{p} \sum_{j=1}^{|G|} \sum_{i=1}^{|S_j|} (\mathbf{x}_{ji} - \hat{\mathbf{m}}_j)(\mathbf{x}_{ji} - \hat{\mathbf{m}}_j)^T,$$

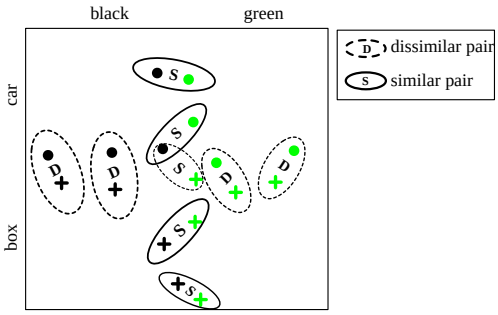
where $\hat{\mathbf{m}}_j$ is the mean of the j-th group of vectors and $|G|$ and $|S_j|$ denote the number of groups and the number of samples in the j-th group, respectively. The matrix $W = \hat{C}^{-\frac{1}{2}}$ is used for transformation and the inverse of $\hat{C}$ as the Mahalanobis matrix. Xing et al. [28] treat the same problem as a convex optimization problem, hence producing local-optima-free algorithms. They present techniques for learning the weighting matrix both for the diagonal and the full matrix case. The major difference between the two approaches lies in the fact that RCA uses closed-form expressions, whereas [28] uses iterative methods that can be sensitive to parameter tuning and are computationally expensive.

C. Aggarwal [3] discusses a systematic framework for designing distance functions sensitive to particular characteristics of the data.

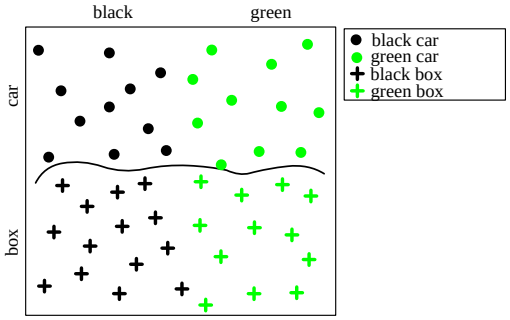
[4]Contextual information is also called side information in some papers such as [7, 28]



(a) Classification boundary of the black and the green



(b) Contextual information labeled by user when query concept is car



(c) Improved classification boundary after using a modified distance function via DMA

Figure 1: An Illustrative Classification Example

The models used are the parametric Minkowski model

$$D(\mathbf{x}_i, \mathbf{x}_j, \boldsymbol{\lambda}) = \left(\sum_{r=1}^{m} \lambda_r |x_{ir} - x_{jr}|^p \right)^{\frac{1}{p}}$$

and the the parametric cosine model

$$\cos(\mathbf{x}_i, \mathbf{x}_j, \boldsymbol{\lambda}) = \sum_{r=1}^{m} \frac{\lambda_r . x_{ir} . x_{jr}}{\sqrt{\sum_{r=1}^{m} x_{ir}^2 \sum_{r=1}^{m} x_{jr}^2}}.$$

Both these models attempt to minimize the error with respect to each λ_r. The parametric Minkowski model can be thought of as feature weighting in $\mathcal{I}$. Similarly, the parametric cosine model can be thought of as the inner product in $\mathcal{I}$.

In summary, metric learning aims to learn a good distance function by computing the optimal feature weightings in the input space $\mathcal{I}$. Clearly, this linear transformation is restrictive in terms of modeling complex semantics. Although one can perform a non-linear transformation on the features via a basis function ϕ in $\mathcal{I}$, such a transformation is *explicit* and the resulting computational complexity renders this approach impractical. The kernel learning approach, which we discuss next, successfully addresses the concern about computational complexity.

2.2 Kernel Learning

Kernel-based methods attempt to *implicitly* map a set of data vectors $\mathcal{X} = \{\mathbf{x}\}_{i=1}^{m}$ in $\mathcal{I}$ to some other high-dimensional (possibly infinite) projected space $\mathcal{P}$, using a basis function (usually nonlinear) ϕ, where the mapped data can be separated by applying a linear procedure [25]. Kernel function K is defined as an inner product between two vectors in projected space $\mathcal{P}$, $\phi(\mathbf{x}_i)$ and $\phi(\mathbf{x}_j)$, as

$$K(\mathbf{x}_i, \mathbf{x}_j) = < \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) > .$$

Kernel-based methods use these inner products (K) as a similarity measure. (Theoretical justifications are presented in [23].) The kernel K provides an elegant way of dealing with nonlinear algorithms by reducing them to linear ones in $\mathcal{P}$. Any algorithm that can be expressed in terms of inner products can be made nonlinear by substituting kernel values for the inner products. A typical example is Support Vector Machines [25].

The requirement for choosing a valid K is that it should be positive (semi-) definite and symmetric. Three popular kernels employed are the polynomial kernel $(< \mathbf{x}_i, \mathbf{x}_j > +1)^p$, the Gaussian radial basis function (RBF) kernel $\exp(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|_2^2}{2\sigma^2})$, and Laplacian RBF kernel $\exp(-\gamma \parallel \mathbf{x}_i - \mathbf{x}_j \parallel_1)$. In $\mathcal{P}$, the distance between $\mathbf{x}_i$ and $\mathbf{x}_j$ can then be computed via the kernel trick

$$\begin{aligned} d(\mathbf{x}_i, \mathbf{x}_j) &= \|\phi(\mathbf{x}_i) - \phi(\mathbf{x}_j)\|_2 \\ &= \sqrt{K(\mathbf{x}_i, \mathbf{x}_i) + K(\mathbf{x}_j, \mathbf{x}_j) - 2K(\mathbf{x}_i, \mathbf{x}_j)}, \end{aligned} \quad (2)$$

where the validity of the kernel K ensures the resulted distance function $d(\mathbf{x}_i, \mathbf{x}_j)$ to be a valid metric.

An important advantage of using kernels stems from the ease of computing the inner product (similarity measure) in $\mathcal{P}$ without actually having to know ϕ.There has been a lot of work [9, 23, 25] in classification, clustering, and regression methods, using the kernel

$K(\mathbf{x}_i, \mathbf{x}_j)$ for indirect computations of similarity measures (and hence the distance $d(\mathbf{x}_i, \mathbf{x}_j)$) in $\mathcal{P}$.

Due to the central role of the kernel, a poor kernel function can lead to significantly poor performance using kernel methods. Instead of choosing a pre-defined kernel for training, many recent efforts aim to learn a kernel from the training data [11, 20]. All these papers are based on the notion of *kernel alignment* proposed by Cristianini et al. [11] to measure the similarity between two kernel functions. Geometrically, when given a set of training instances $\mathcal{X}$, the *alignment* score is defined as the cosine of the angle between the two kernel matrices[5], after flattening the two matrices into vectors. They also proposed the notion of "ideal kernel" (K^*). Suppose $y(\mathbf{x}_i) \in \{1, -1\}$ is the class label of $\mathbf{x}_i$, K^* is defined as

$$K^*(\mathbf{x}_i, \mathbf{x}_j) = \begin{cases} 1, & \text{if } y(\mathbf{x}_i) = y(\mathbf{x}_j), \\ 0, & \text{if } y(\mathbf{x}_i) \neq y(\mathbf{x}_j), \end{cases} \quad (3)$$

which is the target kernel that a given kernel is supposed to be aligned toward. Kernel-alignment uses the alignment score of a given kernel K to the ideal kernel K^* to indicate the degree to which the kernel fits the training data. Moreover, Cristianini et al. [11] prove the connection between the alignment and the generalization performance of the resulting classifier. Basically, their work shows that with a high alignment on the training set, we can expect a good generalization performance of the resulted distance-based classifier.

Since a kernel K defines a pairwise distance from Eqn. 2, kernel learning was recently applied to distance metric learning. Zhang [29] proposed to idealize a given kernel K using $\tilde{K} = K + K^*$, to achieve a good distance metric which is Euclidean- and Fisher-separable. Then, the achieved distance metric was embedded into a new Euclidean space via Multi-dimensional scaling (MDS) [26]. However, this iterative embedding procedure is computationally expensive, and its idealization model might not be optimal. Kwok et al. [20] considered the same problem of metric learning as a convex optimization problem. The approach works in both the input and the kernel-induced projected spaces. They modify the prior kernel using $\tilde{K} = K + \frac{\gamma}{2}K^*$ and derive distance functions using this modified kernel. Since the number of parameters is related to the number of patterns but not to the dimensionality of the patterns, the approach allows feature weighting to be done efficiently in the possibly infinite dimensional projected spaces. However, their learned distance metric cannot be guaranteed to be positive (semi-) definite. Hence, the induced distance function might not be a valid one. Moreover, their kernel transformation model, $\tilde{K} = K + \frac{\gamma}{2}K^*$, is not an optimal one for kernel idealization because $\tilde{K}(\mathbf{x}_i, \mathbf{x}_j) = K(\mathbf{x}_i, \mathbf{x}_j)$ when two instances $\mathbf{x}_i$ and $\mathbf{x}_j$ are similar ($K^*(\mathbf{x}_i, \mathbf{x}_j) = 0$), which means the intra-class similarity scores remain unchanged using this transformation model.

Our DMA algorithm performs a linear transformation on a prior kernel function[6] in a projected space $\mathcal{P}$ when given a set of training data. We theoretically prove that our method leads to a valid distance function. More importantly, we also show that since the optimization is performed in a convex space, the solution obtained is globally optimal. Consequently, given a prior kernel and contextual information, the alignment needs to be performed just once. Our empirical results show that our proposed method outperforms competing methods on a variety of testbeds.

[5]Given a kernel function K and a set of instances $\mathcal{X}$, the kernel matrix (Gram matrix) is the matrix of inner-products of all possible of pairs from $\mathcal{X} \times X$, $\mathbf{K} = [k_{ij}]$, where $k_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$.

[6]The *kernel trick* in Eqn. 2 uniquely links the kernel functions with the distance function. The former (K) provides the pairwise-similarity measurement between two instances, but the latter (d) provides the pairwise-dissimilarity measurement between two instances. Therefore, when we say a transformation on a prior kernel function, it also means a transformation on a prior distance function, vise versa.

3. DISTANCE FUNCTION ALIGNMENT

Let us consider a two-class classification problem with training dataset $\mathcal{X} = \{\mathbf{x}_i, y_i\}_{i=1}^m$, where $\mathbf{x}_i \in \Re^n$ and $y_i \in \{-1, +1\}$. In Section 2.2, we noted that kernel function $K(\mathbf{x}_i, \mathbf{x}_j)$ can be considered as a similarity measure between instances $\mathbf{x}_i$ and $\mathbf{x}_j$. For instance, when an Gaussian or Laplacian RBF function is employed as the kernel, the value of $K(\mathbf{x}_i, \mathbf{x}_j)$ ranges from 0 to 1, where $K(\mathbf{x}_i, \mathbf{x}_j) = 0$ when $\mathbf{x}_i$ and $\mathbf{x}_j$ are infinitely far away (dissimilar) in the input space, and $K(\mathbf{x}_i, \mathbf{x}_j) = 1$ when $\mathbf{x}_i$ and $\mathbf{x}_j$ are definitely close (similar). The choice of a good kernel is equivalent to the choice of a good distance function for measuring similarity. In the next, we first propose a similarity transformation model to improve a prior kernel (Section 3.1). We then use this model to learn a good distance function from the dataset (Section 3.2).

3.1 Similarity Transformation Model

Since a kernel $K(\mathbf{x}, \mathbf{x}')$ defines a pairwise-similarity measure between two instances $\mathbf{x}$ and $\mathbf{x}'$, we propose a similarity transformation model which can increase the kernel value for the similar pairs, but decrease the kernel value for the dissimilar pairs. In this paper, we use a linear transformation model in $\mathcal{P}$, to modify the kernel from K to $\tilde{K}$. Let β_1 and β_2 denote the slopes of transformation curves for dissimilar and similar pairs, respectively. For a given training data $\mathcal{X}$, the kernel matrix $\mathbf{K}$, corresponding to the kernel K, is then modified as follows

$$\tilde{k}_{ij} = \begin{cases} \beta_1 k_{ij}, & \text{if } y_i \neq y_j, \\ \beta_2 k_{ij} + (1 - \beta_2), & \text{if } y_i = y_j. \end{cases} \quad (4)$$

where $0 \leq \beta_1, \beta_2 \leq 1$ and $\tilde{k}_{ij}$ is the ij-th component of the new kernel matrix $\tilde{\mathbf{K}}$. We do not consider a nonlinear transformation in $\mathcal{P}$ for *two* reasons:

1. A non-linear transformation increases the chances of overfitting the training dataset. Usually, kernel methods use a non-linear function as the kernel. Setting a linear transformation in $\mathcal{P}$ on the prior kernel equals finding a non-linear similarity measurement in $\mathcal{I}$. The mission of DMA is to perform adjustment to the prior kernel based on contextual information, rather than to make drastic changes. Overfitting on contextual information will likely incur high prediction variance (with respect to the contextual information collected).
2. A non-linear transformation generally involves a higher computational cost. Moreover, the mapped instances in projected space $\mathcal{P}$ are supposed to be linearly separable [25]. This makes a non-linear transformation unnecessary in $\mathcal{P}$.

In what follows, we prove two important propositions. Proposition 1 demonstrates that under some constraints on β_1 and β_2, our proposed similarity transformation model in Eqn. 4 ensures a valid kernel. Proposition 2 mathematically demonstrates that under the constraints from Proposition 1, the transformed $\tilde{\mathbf{K}}$ in Eqn. 4 guarantees a better alignment to the ideal $\mathbf{K}^*$ in Eqn. 3 than to the prior $\mathbf{K}$.

PROPOSITION 1. *Under the assumption that* $0 \leq \beta_1 \leq \beta_2 \leq 1$, *the transformed kernel* $\tilde{K}$ *is positive (semi-) definite if the prior kernel* K *is positive (semi-) definite.*

PROOF. The transformed kernel $\tilde{K}$ can be written as follows:

$$\tilde{K} = \beta_1 K + (\beta_2 - \beta_1) K \otimes K^* + (1 - \beta_2) K^*, \quad (5)$$

which corresponds to the kernel matrix $\tilde{\mathbf{K}}$, associated with the training set $\mathcal{X}$, in Eqn. 4. If the prior K is positive (semi-) definite, using the fact that the ideal kernel K^* is positive (semi-) definite [11], we can derive that $\tilde{K}$ in Eqn. 5 is also positive (semi-) definite if $0 \leq \beta_1 \leq \beta_2 \leq 1$. Here, we use the closure properties of kernels, viz. the product and summation of valid kernels also give a valid kernel [23]. ■

PROPOSITION 2. *The kernel matrix* $\tilde{\mathbf{K}}$ *of the transformed kernel* $\tilde{K}$ *obtains a better alignment than the prior kernel matrix* $\mathbf{K}$ *to the ideal kernel matrix* $\mathbf{K}^*$, *if* $0 \leq \beta_1 \leq \beta_2 \leq 1$. *Moreover, a smaller* β_1 *or* β_2 *would induce a higher alignment score.*

PROOF. Please refer to Appendix A. ■

Remark. In both propositions, we assume that $\beta_1 \leq \beta_2$. Recall that β_1 and β_2 stand for the slopes of the similarity transformation curves for dissimilar and similar instance-pairs, respectively. This constraint means that we place more emphasis on decreasing similarity value (K) for dissimilar instance-pairs. This constraint is in line with the spirit of achieving maximum distance between dissimilar pairs to keep the separating margin large. ■

3.2 Distance Metric Learning

Here, we first discuss how to use the similarity transformation model defined in last section for distance metric learning (Section 3.2.1). We then show how to generalize the model to unseen instances without overfitting the contextual information (Section 3.2.2).

3.2.1 Distance Formulation

For a prior kernel K, the inner product of two instances $\mathbf{x}_i$ and $\mathbf{x}_j$ is defined as $\phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$ in $\mathcal{P}$. For simplicity, we denote $\phi(\mathbf{x}_i)$ as ϕ_i. The distance between $\mathbf{x}_i$ and $\mathbf{x}_j$ in $\mathcal{P}$ can thus be computed as $d_{ij}^2 = k_{ii} + k_{jj} - 2k_{ij}$, where $k_{ij} = \phi_i^T \phi_j$. Therefore, for the transformed kernel $\tilde{K}$ in Eqn. 4, the corresponding distance $\tilde{d}_{ij}^2 = \tilde{k}_{ii} + \tilde{k}_{jj} - 2\tilde{k}_{ij}$ in $\mathcal{P}$ can be written in terms of d_{ij}^2 as

$$\tilde{d}_{ij}^2 = \begin{cases} \beta_2 d_{ij}^2, & \text{if } y_i = y_j, \\ \beta_1 d_{ij}^2 + 2(1-\beta_2) + (\beta_2 - \beta_1)(k_{ii} + k_{jj}), & \text{if } y_i \neq y_j. \end{cases} \quad (6)$$

In this paper, we suppose $0 \leq K(\mathbf{x}, \mathbf{x}') \leq 1$, and $K(\mathbf{x}, \mathbf{x}') = 1$ if $\mathbf{x} = \mathbf{x}'$. If some kernels do not satisfy this condition, they can be normalized as $\frac{K(\mathbf{x},\mathbf{x}')}{\sqrt{K(\mathbf{x},\mathbf{x})}\sqrt{K(\mathbf{x}',\mathbf{x}')}}$. In this case, we have the distance $d_{ij}^2 = 2 - 2k_{ij}$ which lies between 0 and 2 in $\mathcal{P}$. Eqn. 6 can thus be rewritten as

$$\tilde{d}_{ij}^2 = \begin{cases} \beta_2 d_{ij}^2, & \text{if } y_i = y_j, \\ \beta_1 d_{ij}^2 + 2(1-\beta_1), & \text{if } y_i \neq y_j. \end{cases} \quad (7)$$

Using the property $0 \leq d_{ij} \leq 2$ after normalization and the condition $0 < \beta_1 \leq \beta_2 < 1$, we can see that $\tilde{d}_{ij}^2 < d_{ij}^2$ if the two instances are similar ($y_i = y_j$), and $\tilde{d}_{ij}^2 > d_{ij}^2$ if the two instances are dissimilar ($y_i \neq y_j$). In other words, the transformed distance metric in Eqn. 7 decreases the intra-class pairwise distance in $\mathcal{P}$, and increases the inter-class pairwise distance in $\mathcal{P}$. The developed distance metric (Eqn. 7) is a valid distance metric (non-negativity, symmetry, and triangle inequality) since the transformed kernel in Eqn. 4 is a valid kernel, according to Proposition 1.

Sometimes users may not return the class label y_i for each instance $\mathbf{x}_i$ in the feedback contextual information. Instead, they may only return the information about whether two instances $\mathbf{x}_i$ and $\mathbf{x}_j$ are similar or dissimilar. Denoting the set of pairs of similar instances in the contextual information $\mathcal{X}$ as $\mathcal{S} = \{(\mathbf{x}_i, \mathbf{x}_j) | \mathbf{x}_i \sim \mathbf{x}_j\}$, and the set of pairs of dissimilar instances as $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{x}_j) | \mathbf{x}_i \nsim \mathbf{x}_j\}$, we can extend Eqn. 7 to the case where only $\mathcal{S}$ and $\mathcal{D}$ information is available for classification or clustering

Since the class label information y_i in Eqn 7 can be also considered as a kind of (dis)similarity information by thinking $y_i = y_j$ as $\mathbf{x}_i \sim \mathbf{x}_j$ and $y_i \neq y_j$ as $\mathbf{x}_i \nsim \mathbf{x}_j$, for both cases, we use the notations $\mathcal{S}$ and $\mathcal{D}$ in the rest of the paper as the contextual information from users.

$$\tilde{d}_{ij}^2 = \begin{cases} \beta_2 d_{ij}^2, & \text{if } (\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{S}, \\ \beta_1 d_{ij}^2 + 2(1-\beta_1), & \text{if } (\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{D}. \end{cases} \quad (8)$$

3.2.2 Generalization

To generalize the distance metric in Eqn. 8 to unseen data, we use the feature-weighting method [15] by modifying the inner product $k_{ij} = \phi_i^T \phi_j$ in $\mathcal{P}$ as $\phi_i^T \mathbf{A}\mathbf{A}^T \phi_j$. Here, $\mathbf{A}_{m' \times m'}$ is the weighting matrix and m' is the dimension of $\mathcal{P}$. Based on the idea of feature reduction [15], we aim to achieve a small rank of $\mathbf{A}$, which means that the dimensionality of feature vector ϕ is kept small in projected space $\mathcal{P}$. The corresponding distance function thus becomes

$$\tilde{d}_{ij}^2 = (\phi_i - \phi_j)^T \mathbf{A}\mathbf{A}^T (\phi_i - \phi_j). \quad (9)$$

To solve for $\mathbf{A}\mathbf{A}^T$ so that the distance metric in Eqn. 9 equals the distance in Eqn. 8, we formulate the problem as a convex optimization problem whose objective function is to minimize the rank of the weighting matrix $\mathbf{A}$. However, it induces an NP-Hard problem by directly minimizing $\mathrm{rank}(\mathbf{A})$, the so-called zero-norm problem in [5]. Since minimizing the trace of a matrix tends to give a low-rank solution when the matrix is symmetric [13], in this paper, we approximate the rank of a matrix by its trace. Moreover, since $\mathrm{rank}(\mathbf{A}\mathbf{A}^T) = \mathrm{rank}(\mathbf{A}\mathbf{A}^T\mathbf{A}\mathbf{A}^T) \approx \mathrm{trace}(\mathbf{A}\mathbf{A}^T\mathbf{A}\mathbf{A}^T) = \|\mathbf{A}\mathbf{A}^T\|_{\mathrm{F}}^2$, we approximate the problem of minimizing the rank of $\mathbf{A}$ by minimizing its Frobenius norm $\|\mathbf{A}\mathbf{A}^T\|_{\mathrm{F}}^2$. The corresponding primal problem is formulated as

$$\min_{\mathbf{A}\mathbf{A}^T, \beta_1, \beta_2} \quad \frac{1}{2}\|\mathbf{A}\mathbf{A}^T\|_{\mathrm{F}}^2 + C_{\mathcal{D}}\beta_1 + C_{\mathcal{S}}\beta_2, \quad (10)$$

$$\text{s.t.} \quad \beta_2 d_{ij}^2 = \tilde{d}_{ij}^2, \qquad (\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{S} \quad (11)$$

$$\tilde{d}_{ij}^2 = \beta_1 d_{ij}^2 + 2(1-\beta_1), \qquad (\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{D} \quad (12)$$

$$\beta_2 \geq \beta_1,$$
$$\beta_1 \geq 0,$$
$$1 - \beta_2 \geq 0,$$

where $C_{\mathcal{S}}$ and $C_{\mathcal{D}}$ are two non-negative hyper-parameters. Proposition 2 shows that a large β_1 or β_2 will induce a lower alignment

score. However, on the other hand, $\beta_1 = \beta_2 = 0$ would overfit the training dataset. We hence add two penalty terms $C_{\mathcal{D}}\beta_1$ and $C_{\mathcal{S}}\beta_2$ to control the alignment degree. (We present a method on tuning $C_{\mathcal{S}}$ and $C_{\mathcal{D}}$ in Appendix C) Parameters β_1 and β_2 are related with $\mathbf{A}\mathbf{A}^T$ in the constraints (11) and (12). This strategy is similar to that used in Support Vector Machines [25], which limits the length of weight vector $\|\mathbf{w}\|^2$ in projected space $\mathcal{P}$ to combat the overfitting problem. Compared to the primal formulation used by Xing et al. in [28], we do not introduce one more constraint $\mathbf{A}\mathbf{A}^T \succeq 0$ to guarantee its positive semi-definiteness since the condition is satisfied by the constraints in (11) and (12), as discussed in Section 3.2.1.

The primal formulation above is a constrained convex optimization problem and hence can be solved using the standard method of Lagrange multipliers. (Please refer to Appendix B for the detailed derivation, where we show that the dual formulation of (10) is a standard convex quadratic programming problem with a global optimal solution.) We then obtain $\mathbf{A}\mathbf{A}^T$ as follows:

$$\mathbf{A}\mathbf{A}^{\mathbf{T}} = \sum_{(\mathbf{x}_i,\mathbf{x}_j)\in\mathcal{D}} \alpha_{ij}(\phi_i-\phi_j)(\phi_i-\phi_j)^T - \sum_{(\mathbf{x}_i,\mathbf{x}_j)\in\mathcal{S}} \alpha_{ij}(\phi_i-\phi_j)(\phi_i-\phi_j)^T,$$

where α_{ij} for each instance-pair $(\mathbf{x}_i, \mathbf{x}_j)$ in contextual information is the Lagrange multiplier obtained by solving the dual formulation of (10). (Please see Appendix B for the details).

After solving $\mathbf{A}\mathbf{A}^T$, we then can generalize our distance-metric model defined in Eqn. 9 to the unseen test instances. Suppose $\mathbf{x}$ and $\mathbf{x}'$ are two test instances with unknown class labels, their pairwise distance $\tilde{d}^2(\mathbf{x}, \mathbf{x}')$ after feature weighting is computed as

$$\begin{aligned}\tilde{d}^2(\mathbf{x},\mathbf{x}') &= \left(\phi(\mathrm{x}) - \phi(\mathrm{x}')\right)^T \mathbf{A}\mathbf{A}^T \left(\phi(\mathrm{x}) - \phi(\mathrm{x}')\right) \qquad (14)\\ &= -\sum_{(\mathbf{x}_i,\mathbf{x}_j)\in\mathcal{S}} \alpha_{ij}\left(K_{\mathrm{x}\mathrm{x}_i} - K_{\mathrm{x}\mathrm{x}_j} - K_{\mathbf{x}_i\mathrm{x}'} + K_{\mathbf{x}_j\mathrm{x}'}\right)^2\\ &\quad + \sum_{(\mathbf{x}_i,\mathbf{x}_j)\in\mathcal{D}} \alpha_{ij}\left(K_{\mathrm{x}\mathrm{x}_i} - K_{\mathrm{x}\mathrm{x}_j} - K_{\mathbf{x}_i\mathrm{x}'} + K_{\mathbf{x}_j\mathrm{x}'}\right)^2.\end{aligned}$$

Remark. We note that here our learned distance function $\tilde{d}^2(\mathbf{x}, \mathbf{x}')$ is expressed in terms of the prior kernel K which is chosen before we apply the algorithm. For example, such a prior kernel could be chosen as a Gaussian RBF function $\exp\left(-\frac{\|\mathbf{x}-\mathbf{x}'\|_2^2}{2\sigma^2}\right)$. $K_{\mathrm{x}\mathrm{x}_i}$ and $K_{\mathrm{x}\mathrm{x}_j}$ in Eqn. 13 can thus be computed as $\exp\left(-\frac{\|\mathbf{x}-\mathbf{x}_i\|_2^2}{2\sigma^2}\right)$ and $\exp\left(-\frac{\|\mathbf{x}-\mathbf{x}_j\|_2^2}{2\sigma^2}\right)$, and so are $K_{\mathbf{x}_i\mathrm{x}'}$ and $K_{\mathbf{x}_j\mathrm{x}'}$. Therefore, even in the case where both $\mathbf{x}$ and $\mathbf{x}'$ are unseen test instances, their pairwise distance can still be calculated from Eqn. 13. Moreover, when a linear kernel, $K(\mathbf{x}_i, \mathbf{x}_j) = < \mathbf{x}_i, \mathbf{x}_j >$, is employed, the projected space $\mathcal{P}$ is exactly the original input space $\mathcal{I}$. DMA then becomes a distance-function-learning in $\mathcal{I}$. Therefore, DMA is a general algorithm which can learn a distance function in both $\mathcal{P}$ and $\mathcal{I}$. ■

3.3 Calculation of β_1 and β_2

We identify the support pairs and the non-support pairs using the Lagrange multipliers α_{ij}. We have the following definition.

DEFINITION 1. *A pair of instances $(\mathbf{x}_i, \mathbf{x}_j)$ from $\mathcal{S}$ or $\mathcal{D}$ is called a support pair if it corresponds to a nonzero Lagrange multiplier α_{ij}; otherwise, it is called a non-support pair.*

dataset	# dim	# class	# instance	# training
toy	11	2	100	60
soybean	35	4	47	29
wine	12	3	178	118
glass	10	6	214	145
seg	19	2	210	142
image	144	14	1,897	1,517

Table 1: Datasets Description.

The existence of non-support pairs reduces the computation cost of the distance $\tilde{d}_{ij}^2$ between two unseen instances, since only support pairs with $\alpha_{ij} \neq 0$ contribute in Eqn. 13.

The value of β_1 and β_2 can be computed from the constraints (11) and (12) using the distance between support pairs as follows

$$\beta_1 = \frac{2 - \tilde{d}_{ij}^2}{2 - d_{ij}^2}, \;\; \beta_2 = \frac{\tilde{d}_{ij}^2}{d_{ij}^2}. \qquad (15)$$

As in the case of Support Vector Machines [25], the final β_1 and β_2 are averaged over all support pairs in $\mathcal{D}$ and $\mathcal{S}$, respectively.

4. EXPERIMENTAL EVALUATION

We conducted an extensive empirical study to examine the effectiveness of our context-based distance-function learning algorithm in two aspects.

1. *Contextual information.* We compared the quality of our learned distance function when given quantitatively and qualitatively different contextual information for learning.

2. *Learning effectiveness.* We compared our DMA algorithm with the regular Euclidean metric and Xing et al.'s [28] on classification and clustering applications.

To conduct our experiments, we used six datasets: one toy dataset, four UCI benchmark datasets, and one $2K$ image dataset, which are described as follows:

One toy dataset The *toy* dataset was first introduced in [20]. We used it to compare the effectiveness of our method to other methods. The toy dataset has two classes and eleven features. The first feature of the first class was generated by using the Gaussian distribution $N(3, 1)$, and the first feature of the second class by $N(-3, 1)$; the other ten features were generated by $N(0, 25)$. Each feature was generated independently. Table 1 provides the detailed composition of the *toy* dataset.

Four UCI benchmark datasets The four UCI datasets we experimented with are *soybean*, *wine*, *glass*, and *segmentation* (abbreviated as *seg*). The first three UCI datasets are multi-dimensional. The last dataset, *seg*, is processed as a binary-class dataset by choosing its first class as the target class, and all the other classes as the non-target class. Table 1 presents the detailed description of these four UCI datasets.

2K-image dataset The image dataset was collected from the Corel Image CDs. Corel images have been widely used by the computer vision and image-processing research communities for conducting various experiments. This set contains 2K representative images from fourteen categories, *architecture, bears, clouds, ele-*

phants, fabrics, fireworks, flowers, food, landscape, people, textures, tigers, tools, and *waves*. Each image is represented by a vector of 144 dimensions including color, texture, and shape features [24].

We used k-NN as our classifier, and k-means as our clustering algorithm. When performing classification, we randomly extracted a fraction of the data as training data, and used the remaining data as testing data.For each dataset, the training/testing ratio was empirically chosen via cross-validation so that the classifier using the regular Euclidean metric performs best for a fair comparison. The number of training instances for each dataset is reported in the last column of Table 1. For the k-NN algorithm, we set k to be 5 for non-image datasets and to be 1 for the 2K-image dataset. Our setting is empirically validated as the optimal one for the classifier using the regular Euclidean metric.

For clustering, we used all of the data without splitting them into training and testing pools. The value of k for k-means was set to be the number of classes for each dataset. The contextual-information sets $\mathcal{S}$ (the similar set) and $\mathcal{D}$ (the dissimilar set) were constructed by defining two instances as *similar* if they had the same class label[7], and *dissimilar* otherwise.

We compared four distance functions in the experiments: our distance-function-alignment algorithm (DMA), the Euclidean distance function (Euclidean), the method developed by Kwok et al. [20] (Kwok), and the method developed by Xing et al. [28]. The latter two methods are presented in Section 2. We chose the methods of Kwok et al. and Xing et al. for two reasons. First, both are based on contextual information to learn a distance function, as is used in DMA. Second, the method of Xing et al. is a typical distance-function-learning algorithm in input space $\mathcal{I}$ and has been seen as a yardstick method for learning distance functions in the machine-learning community [7]. The method of Kwok et al. is a new distance-function-learning algorithm in projected space $\mathcal{P}$ developed recently [20]. We compared DMA to both method to test its effectiveness on learning a distance function.

The evaluation procedure is as follows: First, we chose a prior kernel and derived a distance function via the kernel trick. We then ran DMA, Kwok's, and Xing's[8] algorithms, respectively, to learn a new distance function. Finally, we ran k-NN and k-means using the prior and the learned distance functions, and compared their results. Three prior kernels we used in the experiments are linear ($\mathbf{x}^T\mathbf{x}'$), Gaussian ($\exp\left(-\frac{\|\mathbf{x}-\mathbf{x}'\|_2^2}{2\sigma^2}\right)$), and Laplacian ($\exp\left(-\gamma\|\mathbf{x}-\mathbf{x}'\|_1\right)$). For each dataset, we carefully tuned kernel parameters including σ, γ, $C_{\mathcal{S}}$ and $C_{\mathcal{D}}$ via cross-validation. All measurements were averaged over 20 runs.

4.1 Evaluation on contextual information

We chose two datasets, *toy* and *glass*, to examine the performance of our learned distance function when given a different quality or quantity of contextual information.

[7]In this paper, we only consider the case where $\mathcal{S}$ and $\mathcal{D}$ are obtained from the class-label information.

[8]Xing's algorithm cannot be run when the dimensionality of $\mathcal{I}$ is very high or when non-linear kernels, such as Gaussian and Laplacian, are employed. This is because its computational time does not scale well with the high dimensionality of input space and non-linear kernels [20]. We thus did not report the corresponding results in these cases.

Amount		3%	5%	10%	15%	20%
CPU time (seconds)	*toy*	0.03	0.18	0.92	2.01	7.31
	glass	2.56	7.07	61.05	201.2	631.5

Table 2: Ratios of contextual information vs. Computational Cost.

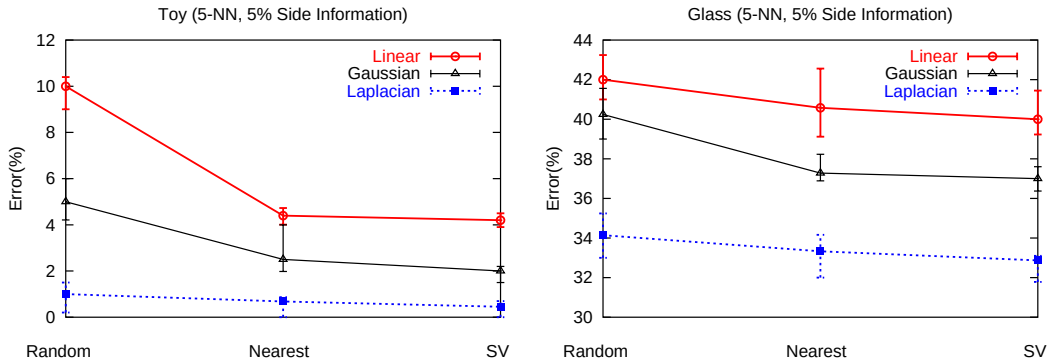
Quality of contextual information We examined two different schemes for choosing contextual information for the k-NN classifier. The first scheme, denoted as *random*, randomly samples a subset of contextual information from $\mathcal{S}$ and $\mathcal{D}$ sets. The second scheme, denoted as *SV*, chooses the most informative instances as contextual information. The most informative instances are those pairs which are located around the decision boundary and hard to classify them to be similar or dissimilar. Additional contextual information provided by the user can help adjust the distance between these pairs. The *SV* scheme uses Support Vector Machines (SVMs) to identify the instances along the class boundary (support vectors), and samples these boundary instances to construct contextual information. Some other strategies can also be employed to help select the boundary instances. We chose SVMs because it is easy to identity the boundary instances (support vectors).

We tested both contextual information selection schemes using our distance-learning algorithm on three prior kernels—linear, Gaussian, and Laplacian. For each scheme, we sampled 5% contextual information from $\mathcal{S}$ and $\mathcal{D}$. Figures 2(a) and (b) present the classification errors (y-axis) of k-NN using the both schemes (x-axis). Figure 2(a) shows the result on the *toy* dataset, and Figure 2(b) shows on the *glass* dataset. We can see that scheme *SV* yielded lower error rates than scheme *random* on both datasets and on all three prior kernels. This shows that choosing the informative contextual information is very useful for learning a good distance function. In the remaining experiments, we employed the *SV* scheme.

Quantity of contextual information We tested the performance of our learned distance function using different amounts of contextual information. Figures 3 (a) and (b) show the classification error rates (y-axis) with different amounts of contextual information (x-axis) available for learning. We ran the k-NN algorithm using the distance metric learned from our algorithm based on different prior kernels. We see that for both *toy* and *glass* datasets, more contextual information is always helpful. However, the improvement on the *glass* dataset did level off after more than 10% contextual information was used.

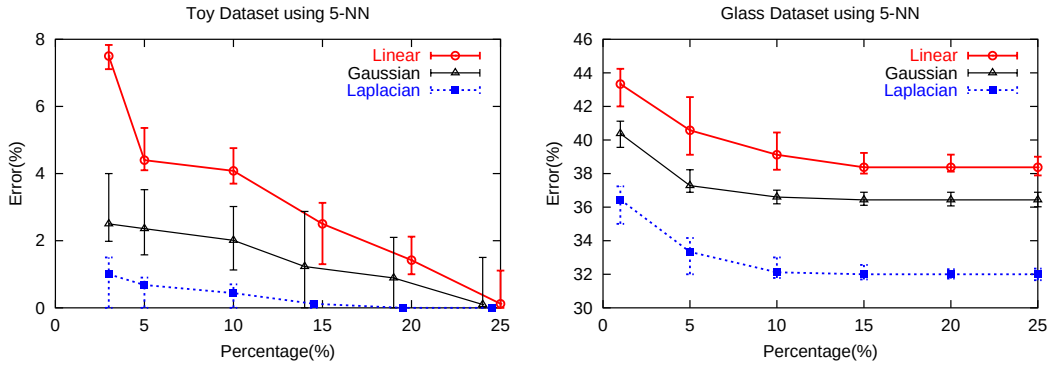
Acquiring contextual information, however, does have its costs. Because contextual information requires user labeling, more contextual information may translate into higher cost. Also, the computational cost of our DMA algorithm is O(n^2), where n represents the amount of contextual information. Using more contextual information increases computational time quadratically. Table 2 reports the CPU time[9] (in seconds) for solving the convex optimization formulation of DMA with different amounts of contextual information. We need to be aware of the tradeoff between the benefit of information gained and the cost of acquisition and computation.

[9]Experiments were done on a Pentium III 1 GHZ workstation with 1 GB DRAM.



(a) Toy (b) Glass

Figure 2: Quality of contextual information vs. Classification Error



(a) Toy (b) Glass

Figure 3: Percentage of contextual information vs. Classification Error

4.2 Evaluation on Effectiveness

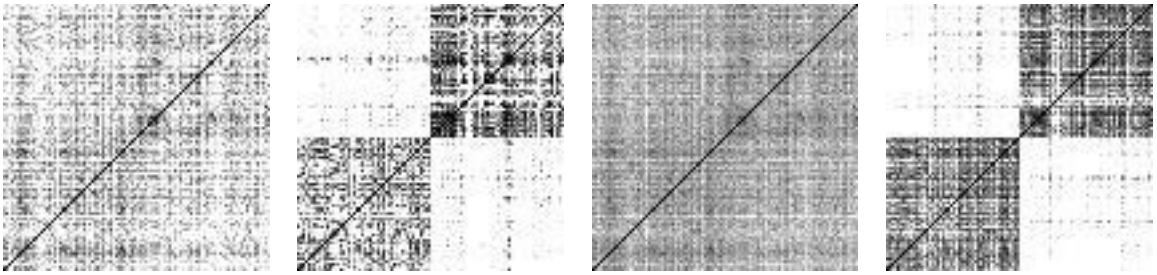
We used the k-NN and k-means algorithms to examine the effectiveness of our distance-learning algorithm on classification and clustering problems. We first report k-NN results, then k-means results.

Classification Results We first used the *toy* dataset to illustrate the effect of DMA on the similarity measures. To set up a baseline to compare *the before* and *the after*, we plotted the pairwise distances of the dataset on a 2D bitmap using two prior distance functions, Gaussian and Laplacian[10]. The pairwise-distance was plotted in the form of a gray-scale bitmap, where the darker a pixel is, the smaller its corresponding distance value will be. We sorted the *toy* dataset such that all instances from the first class were followed by the instances from the second class. Therefore, when we plotted the bitmap, the intra-class distances appeared in the bottom-left quadrant (pairwise distances between the instances of the first class) and the upper-right quadrant (pairwise distances between the instances of the second class), and the inter-class distances appeared in the bottom-right and upper-left quadrants. Ideally, the bottom-left and upper-right quadrants should be darker than the other two quadrants, since the intra-class distances should be shorter than the inter-class distances.

Figure 4(a) shows the Gaussian-distance bitmap and Figure 4(c) the Laplacian-distance bitmap. Both bitmaps appear to be uniformly gray, which means that the intra-class distances and the inter-class distances substantially overlap. Figures 4(b) and (d) plot the distances using the DMA-transformed functions. The figures show that the bottom-left and upper-right quadrants become darker, and the other two lighter. DMA successfully makes intra-class distances shorter and inter-class distances longer. As a result, DMA outperforms the regular k-NN using the Euclidean metric in projected space $\mathcal{P}$ by 7.78% for Gaussian and 9.32% for Laplacian (reported in the caption of Figure 4).

Next, we report the k-NN classification results using five non-image datasets—*toy*, *soybean*, *wine*, *glass*, and *seg*. Table 3 reports all classification error rates (in percentages). We compared our DMA with three other metrics: Euclidean, Kwok, and Xing. For each dataset, we used three different prior kernels—linear, Gaussian, and Laplacian—and then experimented with the four metric candidates. In the third column of the table we report the error rates using the Euclidean distance. The last three columns report the

[10]We use $\sigma = 1.0$ and $\gamma = 0.09$ for Gaussian and Laplacian kernels, respectively



(a) Gaussian (10.00%) (b) DMA (2.22%) (c) Laplacian (10.00%) (d) DMA (0.68%)

Figure 4: Distance-Matrix Comparison on toy dataset using Gaussian and Laplacian Kernels. (b) and (d) show the learned distance matrices using our DMA with a prior Gaussian kernel (a) and a prior Laplacian kernel (c), respectively. The percentage values in the parenthesis are the corresponding classification error rates using 5-NN

Dataset	Kernel	Euclidean	Learned		
			DMA	Kwok	Xing
toy	Linear	10.00	**4.40**	4.80	5.32
	Gaussian	10.00	**2.22**	2.61	-
	Laplacian	10.00	**0.68**	1.10	-
soybean	Linear	2.50	**0.00**	0.12	0.39
	Gaussian	2.50	**1.00**	1.34	-
	Laplacian	2.50	**0.30**	1.19	-
wine	Linear	5.00	**4.86**	4.94	4.98
	Gaussian	5.00	**3.30**	3.50	-
	Laplacian	6.67	**1.65**	1.68	-
glass	Linear	40.58	**39.67**	42.03	40.34
	Gaussian	40.58	**37.28**	47.83	-
	Laplacian	37.68	**33.10**	33.33	-
seg	Linear	2.94	**1.47**	1.51	1.86
	Gaussian	2.94	**0.10**	0.31	-
	Laplacian	**0.00**	**0.00**	2.94	-

Table 3: Classification Error Rates on Non-image Datasets. No results reported for Xing on Gaussian and Laplacian kernels since this algorithm can only work in input space $\mathcal{I}$.

Category	Euclidean	DMA	Kwok
Architecture	29.3	**28.0**	31.7
Bear	64.0	68.0	**63.0**
Cloud	**38.3**	42.1	39.3
Elephant	63.3	**43.9**	60.2
Fabric	**59.0**	67.0	61.0
Firework	14.3	13.3	**12.2**
Flower	45.0	**36.0**	**36.0**
Food	32.5	**29.4**	33.1
Landscape	59.3	**59.2**	58.7
People	52.5	**46.9**	47.0
Texture	38.0	**35.0**	43.0
Tiger	46.7	**37.8**	52.2
Tool	15.0	**12.0**	15.0
Wave	44.3	**39.6**	48.1
total	41.6	**38.9**	41.4

Table 4: Image-dataset Prediction Error Rates.

results of using DMA, Kwok, and Xing, respectively. The best results achieved are shown in bold. First, compared to Euclidean, DMA performs better on almost all datasets, with only one exception being tying on *seg* with Laplacian (both achieved zero classification error rate). Second, DMA outperforms Kwok on all datasets, improving the classification error rates by an average of 0.40%, 0.45%, 0.12%, 5.38%, and 1.06% on the five datasets, respectively. Finally, DMA obtains better results than Xing's in all testing scenarios.

Finally, we report the prediction error-rates using k-NN on the 2K-image dataset in Table 4. In this experiment, we extracted 80% of the images as training data (reported in Table 1), and retained the remaining images as test data. We chose $k = 1$ and prior kernel as Gaussian. Since Xing's algorithm can run only in input space $\mathcal{I}$, not in projected space $\mathcal{P}$, we limited our report to comparing our distance metric with Kwok et al.'s and the regular Euclidean metric and in $\mathcal{P}$. Concerning the computational cost, we chose only 0.6% contextual information randomly from $\mathcal{S}$ and $\mathcal{D}$. (It took 72 seconds of CPU time to solve the corresponding convex optimization formulation.) Among fourteen categories, the distance metric learned by DMA beats the Euclidean in twelve ones and Kwok in eleven ones. It also improves the total prediction error rate by 2.7 and 2.5 percentile points, respectively.

Clustering Results In Table 5, we report the k-means clustering results using the five non-image datasets—*toy*, *soybean*, *wine*, *glass*, and *seg*. To measure the quality of clustering, we used the clustering error rate defined in [28] as follows:

$$\sum_{i>j} \frac{1\{1\{c_i = c_j\} \neq 1\{\hat{c}_i = \hat{c}_j\}\}}{0.5n(n-1)},$$

where $\{c_i\}_{i=1}^n$ denotes the true cluster to which $\mathbf{x}_i$ belongs, $\{\hat{c}_i\}_{i=1}^n$ denotes the cluster predicted by a clustering algorithm, n is the number of instances in the dataset, and $1\{\cdot\}$ the indicator function ($1\{\text{true}\} = 1$, and $1\{\text{false}\} = 0$). From Table 5, we can see that DMA achieves the best clustering results in almost all testing scenarios except for three combinations of the *soybean* and Gaussian,

Dataset	Kernel	Euclidean	Learned		
			DMA	Kwok	Xing
toy	Linear	50.5	**48.2**	48.5	48.9
	Gaussian	50.5	**43.2**	47.1	-
	Laplacian	48.5	**33.5**	38.9	-
soybean	Linear	33.2	**23.0**	24.2	25.8
	Gaussian	32.6	27.0	**22.4**	-
	Laplacian	32.1	**22.8**	34.8	-
wine	Linear	37.1	36.0	**35.8**	36.3
	Gaussian	36.3	**35.7**	35.9	-
	Laplacian	36.3	**26.8**	32.0	-
glass	Linear	31.5	**31.3**	37.0	35.5
	Gaussian	**31.0**	33.6	40.8	-
	Laplacian	31.5	**30.3**	33.3	-
seg	Linear	43.3	**24.2**	25.3	33.0
	Gaussian	40.5	**19.6**	34.4	-
	Laplacian	36.2	20.3	**14.1**	-

Table 5: Clustering Error Rates on Non-image Datasets. Again, no results reported for Xing on Gaussian and Laplacian kernels since this algorithm can only work in input space $\mathcal{I}$.

the *wine* and Linear, and the *glass* and Gaussian. DMA performs better than Xing in all cases where the linear kernel is used.

4.3 Observations

From the experimental results, we make the following observations:

1. *Quality of contextual information counts*. Choosing contextual information with good quality can be very useful for learning a good distance metric, as shown in Figure 2. The best contextual information is the one that can provide maximal information to depict the context. As illustrated in our experiment, the boundary instances tend to be most useful, since when they are disambiguated through additional contextual information, we can achieve the best context-based alignment on the boundary (on the function).

2. *Quantity matters little*. Choosing more contextual information is useful for learning a good distance metric. However, as shown in Figure 3, the cost of increasing quantity can outweigh the benefit, once the quality of information starts to deteriorate.

3. DMA *helps*. DMA improves upon prior functions in almost all of our test scenarios. Occasionally, DMA performs slightly worse than the prior function. We conjecture that this may be caused by overfitting in certain circumstances. Specifically, when some chosen prior kernels may not be the best model for the datasets, and further aligning these kernels could be counter-productive. We will further investigate related issues in our future work.

5. CONCLUSION AND FUTURE WORK

In this paper, we reported our study of an important database issue—formulating context-based distance function for measuring similarity. Our proposed DMA method performs non-linear transformation on a prior distance function in the input space, considering contextual information, without incurring high computational cost. Theoretically, we demonstrated that our method achieves optimal alignment toward the ideal kernel. Empirically, we showed that DMA improves similarity measures and hence leads to improved performance in clustering and classification applications.

Considering contextual information in information retrieval has been identified as an important research area. Our work provides a mechanism that is more effective than the traditional ones because it achieves both model flexibility and computational efficiency. One of our future research tasks is to theoretically analyze the the generalization ability of DMA. Cristianini et al. [11] give a proof on the connection between the "ideal" kernel and the generalization ability of the classifier. This might inspire us a method to analyze DMA. Another future research is to investigate how we might best model context (for an application and for a user), and how we might effectively formulate and gather contextual information to take advantage of of our proposed mechanisms. We also plan to study the effectiveness of DMA on time-sequence data [16], where a data object might not have a vector-space representation.

6. REFERENCES

[1] *NSF Intelligent Data Management (IDM) Principal Investigator Workshop*. SIGMOD Record (to appear), http://www2.cs.washington.edu/nsf2003/, 2003.

[2] C. Aggarwal, A. Hinneburg, and D. Keim. On the surprising behavior of distance metrics in high dimensional spaces. *In Proceedings of International Conference on Database Theory*, pages 420–434, 2001.

[3] C. C. Aggarwal. Towards systematic design of distance functions for data mining applications. *The Ninth ACM SIGKDD International Conference on Knowledge Discovery in Data and Data Mining*, 2003.

[4] M. A. Aizerman, E. M. Braverman, and L. I. Rozonoer. Theoretical foundations of the potential function method in pattern recognition learning. *Automation and Remote Control*, 25:821–837, 1964.

[5] E. Amaldi and V. Kann. On the approximability of minimizing non-zero variables or unsatisfied relations in linear systems. *Theoretical Computer Science*, 209:237–260, 1998.

[6] T. Ashwin, R. Gupta, and S. Ghosal. Adaptable similarity search using non-relevant information. *In Proceedings of the* 28*th VLDB Conference*, August 2002.

[7] A. Bar-hillel, T. Hertz, N. Shental, and D. Weinshall. Learning distance functions using equivalence relations. *In Proceedings of the Twentieth International Conference on Machine Learning*, August 2003.

[8] I. Bartolini, P. Ciacci, and F. Waas. Feedbackbypass: A new approach to interactive similarity query processing. *In Proceedings of the* 27*th VLDB Conference*, pages 201–210, September 2001.

[9] A. Ben-Hur, D. Horn, H. T. Siegelmann, and V. Vapnik. Support vector clustering. *Journal of Machine Learning Research*, 2:125–137, March 2002.

[10] K. Beyer, J. Goldstein, R. Ramakrishnan, and U. Shaft. When is "nearest neighbor" meaningful? *Lecture Notes in Computer Science*, 1999.

[11] N. Cristianini, J. Kandola, A. Elisseeff, and J. Shawe-Taylor. On kernel target alignment. *Journal Machine Learning Research*, 1, 2002.

[12] R. Fagin, R. Kumar, and D. Sivakumar. Efficient similarity search and classification via rank aggregation. *In Proceedings of ACM SIGMOD Conference on Management of Data*, pages 301–312, June 2003.

[13] M. Fazel. Matrix rank minimization with applications. *Ph.D. Thesis, Electrical Engineering Dept, Stanford University*, March 2002.

[14] A. Gionis, P. Indyk, and R. Motwani. Similarity search in high dimensions via hashing. *In Proceedings of the 25th VLDB Conference*, pages 518–529, 1999.

[15] Y. Grandvalet and S. Canu. Adaptive scaling for feature selection in svms. *Advances in Neural Information Processing Systems 15*, 2003.

[16] E. Keogh, K. Chakrabarti, M. Pazzani, and S. Mehrotra. Locally adaptive dimensionality reduction for indexing large time series databases. *In Proceedings ACM SIGMOD Conference on Management of Data*, May 2001.

[17] D.-H. Kim and C.-W. Chung. Qcluster: Relevance feedback using adaptive clustering for content-based image retrieval. *In Proceedings of ACM SIGMOD Conference on Management of Data*, June 2003.

[18] H. P. Kriegel, P. Krger, M. Pfeifle, M. Schubert, and S. Brecheisen. Using sets of feature vectors for similarity search on voxelized cad objects. *In Proceedings of ACM SIGMOD Conference on Management of Data*, June 2003.

[19] H. W. Kuhn and A. W. Tucker. Nonlinear programming. *In Proceedings of the Second Berkeley Symposium on Mathematical Statistics and Probabilistics*, pages 481–492, 1951.

[20] J. T. Kwok and I. W. Tsang. Learning with idealized kernels. *In Proceedings of the Twentieth International Conference on Machine Learning*, pages 400–407, August 2003.

[21] M. Nadler and E. P. Smith. *Pattern Recognition Engineering*. New York: Wiley, 1993.

[22] Y. Rui and T. Huang. Optimizing learning in image retrieval. *IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pages 236–245, June 2000.

[23] B. Schölkopf and A. Smola. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press, Cambridge, MA, 2002.

[24] S. Tong and E. Chang. Support vector machine active learning for image retrieval. *In Proceedings of ACM International Conference on Multimedia*, pages 107–118, 2001.

[25] V. Vapnik. *Statistical Learning Theroy*. John Wiley and Sons, 1998.

[26] A. R. Webb. Multidimensional scaling by iterative majorization using radial basis functions. *Pattern Recognition*, 28(5):753–759, 1995.

[27] D. Wettschereck, D. Aha, and T. Mohri. A review and empirical evaluation of feature weighting methods for a class of lazy learning algorithms. *Artificial Intelligence Review*, 11:273–314, 1997.

[28] E. Xing, A. Ng, M. Jordan, and S. Russell. Distance metric learning, with application to clustering with side-information. *Advances in Neural Information Processing Systems 15*, 2003.

[29] Z. Zhang. Learning metrics via discriminant kernels and multidimensional scaling : Towards expected euclidean representation. *In Proceedings of the Twentieth International Conference on Machine Learning*, August 2003.

APPENDIX

A. PROOF OF PROPOSITION 2

Proposition 2 *The kernel matrix $\tilde{\mathbf{K}}$ of the idealized kernel $\tilde{K}$ obtains a better alignment than the prior kernel matrix $\mathbf{K}$ to the ideal kernel matrix $\mathbf{K}^*$, if $0 \leq \beta_1 \leq \beta_2 \leq 1$. Moreover, a smaller β_1 or β_2 will result in a higher alignment score.*

PROOF. In [20], it has been proven that a kernel with the following form has a higher alignment score with the ideal kernel K^* than the original kernel K,

$$\overline{K} = K + \gamma K^*, \quad \gamma > 0, \tag{16}$$

where we use $\overline{K}$ to distinguish with our $\tilde{K}$ defined in Eqn. 4. According to the definition of kernel target alignment [11], we have

$$\left[\frac{< \mathbf{K}, \mathbf{K}^* >}{\sqrt{< \mathbf{K}, \mathbf{K} >}}\right]^2 < \left[\frac{< \mathbf{K} + \gamma\mathbf{K}^*, \mathbf{K}^* >}{\sqrt{< \mathbf{K} + \gamma\mathbf{K}^*, \mathbf{K} + \gamma\mathbf{K}^* >}}\right]^2, \tag{17}$$

where the common item $\sqrt{< \mathbf{K}^*, \mathbf{K}^* >}$ is omitted at both sides of inequality. We then expand (16). Employing Eqn. 3 and Eqn. 15, we achieve

$$\frac{\left(\sum_{\mathcal{S}} k_{ij}\right)^2}{\sum_{\mathcal{S}} k_{ij}^2 + \sum_{\mathcal{D}} k_{ij}^2} < \frac{\left[\sum_{\mathcal{S}} (k_{ij} + \gamma)\right]^2}{\sum_{\mathcal{S}} (k_{ij} + \gamma)^2 + \sum_{\mathcal{D}} k_{ij}^2} \tag{18}$$

Defining $\gamma = \frac{1-\beta_2}{\beta_2} > 0$, where β_2 is the parameter in Eqn. 4, we then rewrite the right side in (17) as follows

$$\begin{aligned}
& \frac{\left[\sum_{\mathcal{S}} (k_{ij} + \frac{1-\beta_2}{\beta_2})\right]^2}{\sum_{\mathcal{S}} (k_{ij} + \frac{1-\beta_2}{\beta_2})^2 + \sum_{\mathcal{D}} k_{ij}^2} \\
= & \frac{\beta_2^2 \left[\sum_{\mathcal{S}} (k_{ij} + \frac{1-\beta_2}{\beta_2})\right]^2}{\beta_2^2 \sum_{\mathcal{S}} (k_{ij} + \frac{1-\beta_2}{\beta_2})^2 + \beta_2^2 \sum_{\mathcal{D}} k_{ij}^2} \\
= & \frac{\left[\sum_{\mathcal{S}} (\beta_2 k_{ij} + (1-\beta_2))\right]^2}{\sum_{\mathcal{S}} (\beta_2 k_{ij} + (1-\beta_2))^2 + \beta_2^2 \sum_{\mathcal{D}} k_{ij}^2} \qquad (19) \\
\leq & \frac{\left[\sum_{\mathcal{S}} (\beta_2 k_{ij} + (1-\beta_2))\right]^2}{\sum_{\mathcal{S}} (\beta_2 k_{ij} + (1-\beta_2))^2 + \beta_1^2 \sum_{\mathcal{D}} k_{ij}^2} \qquad (20) \\
= & \left[\frac{< \tilde{\mathbf{K}}, \mathbf{K}^* >}{\sqrt{< \tilde{\mathbf{K}}, \tilde{\mathbf{K}} >}}\right]^2, \qquad (21)
\end{aligned}$$

where we apply the assumption of $\beta_1 \leq \beta_2$ from (18) to (19), and employ Eqn. 4 in the last step (20). Combining the inequations (16)

and (20), we obtain

$$\frac{< \mathbf{K}, \mathbf{K}^* >}{\sqrt{< \mathbf{K}, \mathbf{K} >}\sqrt{< \mathbf{K}^*, \mathbf{K}^* >}} < \frac{< \tilde{\mathbf{K}}, \mathbf{K}^* >}{\sqrt{< \tilde{\mathbf{K}}, \tilde{\mathbf{K}} >}\sqrt{< \mathbf{K}^*, \mathbf{K}^* >}}.$$

Therefore, the idealized kernel in Eqn. 4 can achieve better alignment than the original kernel K under the assumption of $0 \leq \beta_1 \leq \beta_2 \leq 1$. Moreover, a greater γ in Eqn. 15 will have a higher alignment [20]. Recall that $\beta_2 = \frac{1}{1+\gamma}$. Hence, a smaller β_2 will have a higher alignment. On the other hand, from (19) and (20), we can see that the alignment score of $\tilde{K}$ is an decreasing function w.r.t. β_1. Therefore, a smaller β_1 or β_2 will result in a higher alignment score. ■

B. DERIVATION OF DISTANCE FUNCTION ALIGNMENT

The primal problem of DMA can be formulated as

$$\min_{\mathbf{A}\mathbf{A}^\mathbf{T}, \beta_1, \beta_2} \quad \frac{1}{2}\|\mathbf{A}\mathbf{A}^\mathbf{T}\|_F^2 + C_\mathcal{D}\beta_1 + C_\mathcal{S}\beta_2,$$

$$\text{s.t.} \quad \beta_2 d_{ij}^2 = \tilde{d}_{ij}^2, \qquad (\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{S} \tag{22}$$

$$\tilde{d}_{ij}^2 = \beta_1 d_{ij}^2 + 2(1-\beta_1), \qquad (\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{D} \tag{23}$$

$$\beta_2 \geq \beta_1, \tag{24}$$

$$\beta_1 \geq 0,$$

$$1 - \beta_2 \geq 0,$$

where $C_\mathcal{S}$ and $C_\mathcal{D}$ are two non-negative hyper-parameters. Proposition 2 shows that a large β_1 or β_2 will induce a lower alignment score. However, on the other hand, $\beta_1 = \beta_2 = 0$ would overfit the training dataset. We hence add two penalty terms $C_\mathcal{D}\beta_1$ and $C_\mathcal{S}\beta_2$ to control the alignment degree. (We present the methodology on tuning $C_\mathcal{S}$ and $C_\mathcal{D}$ in Appendix C.) β_1 and β_2 are related with $\mathbf{A}\mathbf{A}^T$ in the constraints (21) and (22). This strategy is similar to that used in SVMs, which limits the length of weight vector $\|\mathbf{w}\|^2$ in projected space $\mathcal{P}$ to combat the overfitting problem. Compared to the primal formulation used by Xing et al. in [28], we do not introduce one more constraint $\mathbf{A}\mathbf{A}^T \succeq 0$ to guarantee its positive semi-definiteness since the condition is satisfied by the constraints in (21) and (22), as discussed in Section 3.2.1.

The constrained optimization problem above can be solved by considering the corresponding Lagrangian formulation

$$\begin{aligned}
&\mathcal{L}(\mathbf{A}\mathbf{A}^\mathbf{T}, \beta_1, \beta_2, \boldsymbol{\eta}, \mu, \gamma, \pi) \qquad (25)\\
&= \frac{1}{2}\|\mathbf{A}\mathbf{A}^\mathbf{T}\|_F^2 + C_\mathcal{D}\beta_1 + C_\mathcal{S}\beta_2\\
&- \sum_{(\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{S}} \alpha_{ij} \left[(\phi_i - \phi_j)^T(\beta_2 \mathbf{I} - \mathbf{A}\mathbf{A}^T)(\phi_i - \phi_j)\right]\\
&- \sum_{(\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{D}} \alpha_{ij} \left[(\phi_i - \phi_j)^T(\mathbf{A}\mathbf{A}^T - \beta_1 \mathbf{I})(\phi_i - \phi_j) - 2(1-\beta_1)\right]\\
&- \mu\beta_1 - \gamma(\beta_2 - \beta_1) - \pi(1-\beta_2),
\end{aligned}$$

where the Lagrangian multipliers (dual variables) $\alpha_i, \mu, \gamma, \pi \geq 0$. This function has to be minimized w.r.t. the primal variables $\mathbf{A}\mathbf{A}^\mathbf{T}, \beta_1, \beta_2$, and maximized w.r.t. the dual variables $\boldsymbol{\alpha}, \boldsymbol{\eta}, \mu, \gamma, \pi$. To eliminate the primal variables, we set the corresponding partial derivatives to be zero, obtaining the following conditions:

$$\begin{aligned}
\mathbf{A}\mathbf{A}^\mathbf{T} &= -\sum_{(\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{S}} \alpha_{ij}(\phi_i - \phi_j)(\phi_i - \phi_j)^T\\
&\quad + \sum_{(\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{D}} \alpha_{ij}(\phi_i - \phi_j)(\phi_i - \phi_j)^T, \qquad (26)\\
C_\mathcal{S} + \pi - \gamma &= \sum_{(\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{S}} \alpha_{ij}(\phi_i - \phi_j)^T(\phi_i - \phi_j), \qquad (27)\\
C_\mathcal{D} + \gamma - \mu &= \sum_{(\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{D}} \alpha_{ij}\left[2 - (\phi_i - \phi_j)^T(\phi_i - \phi_j)\right], \qquad (28)
\end{aligned}$$

Substituting the conditions of (26) and (27) into (24), we obtain the following dual formulation

$$\begin{aligned}
\mathcal{W}(\boldsymbol{\alpha}, \pi) =& \frac{1}{2}\|\mathbf{A}\mathbf{A}^\mathbf{T}\|_F^2 + 2\sum_{(\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{D}} \alpha_{ij}\\
&- \sum_{(\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{D}} \alpha_{ij}(\phi_i - \phi_j)^T\mathbf{A}\mathbf{A}^T(\phi_i - \phi_j)\\
&+ \sum_{(\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{S}} \alpha_{ij}(\phi_i - \phi_j)^T\mathbf{A}\mathbf{A}^T(\phi_i - \phi_j)\\
&- \pi, \qquad (29)
\end{aligned}$$

which has to be maximized w.r.t. α_{ij}'s and $\pi \geq 0$. Actually, π can be removed from the dual function (28), since $\frac{\partial \mathcal{W}(\boldsymbol{\alpha}, \pi)}{\partial \pi} = -1 < 0$, which means that $\mathcal{W}(\boldsymbol{\alpha}, \pi)$ is a decreasing function w.r.t π. Hence, $\mathcal{W}(\boldsymbol{\alpha}, \pi)$ is maximal at $\pi = 0$.

Now, the dual formulation (28) becomes a convex quadratic function w.r.t. only α_{ij}. Next, we examine the constraints of dual formulation on α_{ij}. According to the KKT theorem [19], we have the following conditions

$$\begin{aligned}
\alpha_{ij}\left[\beta_2 d_{ij}^2 - \tilde{d}_{ij}^2\right] &= 0, \quad (\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{S} \qquad (30)\\
\alpha_{ij}\left[\tilde{d}_{ij}^2 - \beta_1 d_{ij}^2 - 2(1-\beta_1)\right] &= 0, \quad (\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{D} \qquad (31)\\
\gamma(\beta_2 - \beta_1) &= 0, \qquad (32)\\
\mu\beta_1 &= 0, \qquad (33)\\
\pi(1-\beta_2) &= 0. \qquad (34)
\end{aligned}$$

The constraint (23) limits $\beta_2 \geq \beta_1$. In the case of $\beta_1 = \beta_2 = 0$, the training dataset would be overfitted. In addition, in the case of $\beta_1 = \beta_2 = 1$, $\tilde{d}_{ij}^2$ is exactly equal to the original distance metric d_{ij}^2 but we do not get any improvement. To avoid these cases, we then change (23) to be a strict inequality constraint of $\beta_2 > \beta_1$. Therefore, we have $\gamma = 0$ from (31). Using the properties of $\gamma = 0$ and $\pi = 0$, we can then change the dual formulation (28) and its constraints of (26) and (27) by substituting the expansion form of

$\mathbf{AA^T}$ in (25) as follows:

$$\begin{aligned}
&\max_{\boldsymbol{\alpha}} \quad \mathcal{W}(\boldsymbol{\alpha}) \\
&= 2 \sum_{(\mathbf{x}_i,\mathbf{x}_j)\in\mathcal{D}} \alpha_{ij} \\
&- \frac{1}{2} \sum_{(\mathbf{x}_i,\mathbf{x}_j)\in\mathcal{S}} \sum_{(\mathbf{x}_k,\mathbf{x}_l)\in\mathcal{S}} \alpha_{ij}\alpha_{kl} \left((\phi_i-\phi_j)^T(\phi_k-\phi_l)\right)^2 \\
&- \frac{1}{2} \sum_{(\mathbf{x}_i,\mathbf{x}_j)\in\mathcal{D}} \sum_{(\mathbf{x}_k,\mathbf{x}_l)\in\mathcal{D}} \alpha_{ij}\alpha_{kl} \left((\phi_i-\phi_j)^T(\phi_k-\phi_l)\right)^2 \\
&+ \sum_{(\mathbf{x}_i,\mathbf{x}_j)\in\mathcal{S}} \sum_{(\mathbf{x}_k,\mathbf{x}_l)\in\mathcal{D}} \alpha_{ij}\alpha_{kl} \left((\phi_i-\phi_j)^T(\phi_k-\phi_l)\right)^2, \quad (35)
\end{aligned}$$

$$\text{s.t.} \quad \begin{cases} C_{\mathcal{S}} & = \sum_{(\mathbf{x}_i,\mathbf{x}_j)\in\mathcal{S}} \alpha_{ij}(\phi_i-\phi_j)^T(\phi_i-\phi_j), \\ C_{\mathcal{D}} & \geq \sum_{(\mathbf{x}_i,\mathbf{x}_j)\in\mathcal{D}} \alpha_{ij}\left[2-(\phi_i-\phi_j)^T(\phi_i-\phi_j)\right], \\ 0 & \leq \alpha_{ij}. \end{cases} \quad (36)$$

where $\phi_i^T\phi_j = K(\mathbf{x}_i,\mathbf{x}_j)$ from the kernel trick. The dual formulation (34) is very similar to that of C-SVMs [25]. It is a standard convex quadratic programming, which can result in a global optimal solution without any local minima.

C. ANALYSIS OF DMA ALGORITHM

We analyze our algorithm in two aspects. First, we interpret the relationship of the rank of $\mathbf{AA}^T$ with the Lagrangian multipliers α_{ij}. Next, we present a method for tuning parameters $C_{\mathcal{S}}$ and $C_{\mathcal{D}}$.

C.1 Interpretation of $\mathbf{AA}^T$

Eqn. 13 tells us the relationship of $\mathbf{AA}^T$ with the Lagrangian multipliers α_{ij}'s. We can interpret $\mathrm{rank}(\mathbf{A})$ with respect to α_{ij}, using the following proposition.

PROPOSITION 3. *The total number of support pairs from both $\mathcal{S}$ and $\mathcal{D}$ provides an upper bound on the rank of feature weighting matrix $\mathbf{A}$.*

PROOF. Using the fact that $\mathrm{rank}(\mathbf{B_1}+\mathbf{B_2}) \leq \mathrm{rank}(\mathbf{B_1}) + \mathrm{rank}(\mathbf{B_2})$, where $\mathbf{B_1}$ and $\mathbf{B_2}$ are two arbitrary matrices, we have the following deduction from Eqn. 13

$$\begin{aligned}
\mathrm{rank}(\mathbf{AA}^T) \leq & \sum_{(\mathbf{x}_i,\mathbf{x}_j)\in\mathcal{S},\alpha_{ij}>0} \mathrm{rank}(-\mathbf{M}_{ij}) \\
& + \sum_{(\mathbf{x}_i,\mathbf{x}_j)\in\mathcal{D},\alpha_{ij}>0} \mathrm{rank}(\mathbf{M}_{ij}), \quad (37)
\end{aligned}$$

where $\mathbf{M}_{ij} = \alpha_{ij}(\phi_i-\phi_j)(\phi_i-\phi_j)^T$. Since $(\phi_i-\phi_j)$ is a vector, $\mathrm{rank}(-\mathbf{M}_{ij}) = \mathrm{rank}(\mathbf{M}_{ij}) = 1$. (36) is rewritten as

$$\begin{aligned}
&\mathrm{rank}(\mathbf{A}) = \mathrm{rank}(\mathbf{AA}^T) \leq \\
&\#\{(\mathbf{x}_i,\mathbf{x}_j)\in\mathcal{S}, \alpha_{ij}>0\} + \#\{(\mathbf{x}_i,\mathbf{x}_j)\in\mathcal{D}, \alpha_{ij}>0\}.
\end{aligned}$$

Therefore, the rank of feature weighting matrix $\mathbf{A}$ is bounded above by the total number of support pairs in both $\mathcal{S}$ and $\mathcal{D}$. ■

C.2 Parameter Tuning

Recall the constraints (35) on $C_{\mathcal{S}}$ and $C_{\mathcal{D}}$,

$$C_{\mathcal{S}} = \sum_{(\mathbf{x}_i,\mathbf{x}_j)\in\mathcal{S}} \alpha_{ij} d_{ij}^2, \quad C_{\mathcal{D}} \geq \sum_{(\mathbf{x}_i,\mathbf{x}_j)\in\mathcal{D}} \alpha_{ij}(2-d_{ij}^2).$$

Suppose we decrease $C_{\mathcal{S}}$. The average value α_{ij} of support pairs in $\mathcal{S}$ would be lowered. Hence, a larger $\tilde{d}_{ij}^2$ would be achieved from (13). Since $\beta_1 = \frac{2-\tilde{d}_{ij}^2}{2-d_{ij}^2}$ and $\beta_2 = \frac{\tilde{d}_{ij}^2}{d_{ij}^2}$, we can see that β_1 is a decreasing function w.r.t. $\tilde{d}_{ij}^2$, and β_2 is an increasing function w.r.t. $\tilde{d}_{ij}^2$. Therefore, decreasing $C_{\mathcal{S}}$ would asymptotically decrease β_1.

In addition, if we decrease $C_{\mathcal{D}}$, it means that we lower the upper bound on α_{ij} in $\mathcal{D}$. We can thus expect a smaller average value α_{ij} for support pairs in $\mathcal{D}$. Hence, from Eqn.(13) and (14) , decreasing $C_{\mathcal{D}}$ would asymptotically decrease β_2. Our extensive experiments have confirmed these two phenomena. We thus can use these two properties to guide the parameter tuning, since $\beta_2 \geq \beta_1$ will result in a higher alignment score (from Proposition 2).